
Sebenta 01

Fundamentos de Infraestruturas de Computação na Nuvem

Infraestruturas de Computação na Nuvem | Aula 01

Como usar esta sebenta

Este documento acompanha os slides projetados na aula. Cada página seguinte reproduz um slide e junta, logo a seguir, o texto que desenvolve por extenso a matéria resumida nesse slide. Esta primeira aula é conceptualmente densa, não introduz ainda tecnologias concretas, introduz antes o vocabulário com que todas as aulas seguintes vão ser discutidas. Vale a pena lê-la com calma antes da Aula 02.

Fundamentos de Infraestruturas de Computação na Nuvem

Infraestruturas de Computação na Nuvem, Aula 01

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Esta é a primeira aula da unidade curricular e tem uma função diferente das restantes. Não introduz ainda tecnologias concretas, introduz antes o vocabulário conceptual com que todas as aulas seguintes vão ser discutidas: o que é um recurso partilhado, o que significa virtualizar algo, que atributos tornam um sistema fiável e o que distingue, afinal, uma infraestrutura qualquer de uma infraestrutura de cloud. Esta sebenta segue a mesma ordem dos slides e desenvolve por escrito o raciocínio que normalmente acompanha cada transparência quando é apresentada em voz alta na aula.

Parte 1

O objeto de estudo

A primeira parte da aula pergunta, com alguma insistência, o que é afinal uma infraestrutura e porque é que partilhar recursos entre utilizadores que não confiam uns nos outros é um problema tão antigo quanto difícil de resolver bem. É também aqui que aparece o conceito central do semestre inteiro: a virtualização, entendida não como uma tecnologia específica mas como um princípio geral de desenho de sistemas.

O que distingue uma infraestrutura

- Uma infraestrutura caracteriza-se por ser **partilhada, persistente e transparente em funcionamento normal**.
- Partilhada: serve múltiplos consumidores que não coordenam entre si.
- Persistente: a sua escala de tempo é superior à das aplicações que suporta.
- Transparente: quando funciona bem, ninguém repara nela. Torna-se visível apenas na falha – o que tem consequências para a forma como é financiada e valorizada.

O ponto de partida da aula é definir com cuidado uma palavra que se usa com demasiada leveza no dia a dia: infraestrutura. Três propriedades distinguem uma infraestrutura de um sistema qualquer. É partilhada, no sentido de servir vários consumidores que não coordenam entre si nem sabem, muitas vezes, da existência uns dos outros. É persistente, porque a sua escala de tempo de vida ultrapassa largamente a das aplicações individuais que correm por cima dela, um servidor sobrevive a dezenas de versões do software que nele executa. E é transparente quando funciona bem, o que tem uma consequência curiosa: uma infraestrutura só se torna visível quando falha. Ninguém repara na rede elétrica enquanto ela funciona, e é exatamente essa invisibilidade em funcionamento normal que torna difícil justificar o investimento nela até ao dia em que deixa de funcionar.

O problema

- Existe um conjunto *finito* de recursos físicos e um conjunto *variável* de procuras concorrentes sobre eles.
- Toda a disciplina decorre da tensão entre estes dois factos.
- As perguntas que atravessam o semestre são sempre variações da mesma: *como se reparte um recurso escasso entre consumidores que não confiam uns nos outros, de forma eficiente, previsível e resistente a falhas?*

Partilhar sem interferir.

Esta transparência tem uma origem concreta: existe sempre um conjunto finito de recursos físicos, processadores, memória, largura de banda, e um conjunto de procuras concorrentes sobre esses mesmos recursos que varia ao longo do tempo e que ninguém consegue prever com exatidão. Toda a disciplina que se segue ao longo do semestre nasce desta tensão simples entre o finito e o variável. As perguntas que vão reaparecer, aula após aula, são sempre a mesma pergunta com roupagens diferentes: como se reparte um recurso escasso entre consumidores que não confiam uns nos outros, de forma eficiente, previsível e resistente a falhas? A frase que resume a ideia chave desta parte é curta e vale a pena memorizar: partilhar sem interferir.

Tipos de recurso

- **Recurso:** entidade finita cujo consumo por um utilizador reduz a disponibilidade para os restantes.
- Distinguem-se pela forma como se esgotam e se recuperam:
 - **Renováveis por tempo** – processador, largura de banda. Não usar num instante é desperdício irrecoverável.
 - **Ocupacionais** – memória, espaço em disco. Mantêm-se ocupados até serem libertados.
- Esta distinção determina que mecanismo de partilha é aplicável a cada recurso.

Nem todos os recursos se comportam da mesma forma quando são partilhados, e essa diferença determina que mecanismo de partilha se pode aplicar a cada um. Um recurso, no sentido técnico usado aqui, é uma entidade finita cujo consumo por um utilizador reduz necessariamente a disponibilidade desse recurso para os restantes utilizadores. Os recursos renováveis por tempo, como o processador ou a largura de banda de rede, têm uma propriedade particular: não usar o recurso num dado instante é um desperdício que nunca mais se recupera, esse segundo de processador que ficou parado não volta a existir. Os recursos ocupacionais, como memória ou espaço em disco, comportam-se de forma diferente: mantêm-se ocupados indefinidamente até serem explicitamente libertados por quem os está a usar.

Dois mecanismos de partilha

- **Multiplexagem temporal:** o recurso é atribuído integralmente a um consumidor de cada vez, alternando no tempo. Aplica-se a recursos renováveis por tempo.
- **Multiplexagem espacial:** o recurso é dividido em porções atribuídas simultaneamente. Aplica-se a recursos ocupacionais.
- A comutação temporal tem um custo de *troca de contexto*; a divisão espacial tem um custo de *fragmentação*.
- Praticamente todos os mecanismos que estudaremos são combinações destes dois, aplicadas a níveis diferentes de abstração.

Correspondendo aos dois tipos de recurso descritos no slide anterior, existem dois mecanismos fundamentais de partilha. A multiplexagem temporal atribui o recurso integralmente a um consumidor de cada vez, alternando entre consumidores ao longo do tempo, e aplica-se naturalmente a recursos renováveis por tempo como o processador. A multiplexagem espacial divide o recurso em porções que são atribuídas em simultâneo a vários consumidores diferentes, e aplica-se a recursos ocupacionais como a memória. Cada mecanismo tem um custo próprio: a comutação temporal paga o preço da troca de contexto, o tempo gasto a guardar o estado de um consumidor e a carregar o estado do seguinte, enquanto a divisão espacial paga o preço da fragmentação, espaço que fica reservado mas que nenhum consumidor consegue efetivamente usar. Praticamente todos os mecanismos estudados ao longo do semestre são combinações destes dois princípios, aplicadas a níveis de abstração diferentes, do processador ao armazenamento em disco.

Multiplexagem temporal e espacial



O diagrama compara visualmente os dois mecanismos descritos no slide anterior. Do lado da multiplexagem temporal, repare como o recurso inteiro pertence a um único consumidor em cada intervalo, e é o eixo horizontal do tempo que organiza o acesso de todos. Do lado da multiplexagem espacial, o recurso está fisicamente dividido em partes, cada uma atribuída a um consumidor diferente, e todos acedem em simultâneo às suas respectivas partes. Vale a pena guardar bem esta imagem, porque vai reaparecer, de formas disfarçadas, sempre que se discutir como um processador reparte tempo entre processos ou como a memória de um servidor se reparte entre máquinas virtuais.

Virtualizar não é só criar VMs

- **Virtualizar** é apresentar a um consumidor uma interface que se comporta como um recurso dedicado, sendo na realidade uma parte de um recurso compartilhado.
- O conceito não se restringe a máquinas virtuais: memória virtual, sistemas de ficheiros e endereçamento de rede são todos instâncias do mesmo princípio.
- Requer três capacidades: *traduzir* referências virtuais em físicas, *arbitrar* o acesso concorrente e *isolar* os consumidores entre si.
- Quando qualquer uma das três falha, a abstração deixa de se sustentar – e o comportamento torna-se difícil de explicar a partir da interface.

Chegamos aqui ao conceito talvez mais importante de toda a unidade curricular, e por isso vale a pena defini-lo com precisão. Virtualizar é apresentar a um consumidor uma interface que se comporta como se fosse um recurso dedicado exclusivamente a ele, sendo na realidade apenas uma parte de um recurso maior, compartilhado com outros. Note-se que o conceito não se limita a máquinas virtuais, que serão o exemplo mais estudado nas próximas aulas. A memória virtual de um sistema operativo, um sistema de ficheiros ou o esquema de endereçamento de uma rede são todos instâncias do mesmo princípio geral. Para que a virtualização funcione, são precisas três capacidades em simultâneo: traduzir as referências virtuais que o consumidor usa em referências físicas reais, arbitrar o acesso concorrente quando vários consumidores pedem o mesmo recurso físico ao mesmo tempo, e isolar os consumidores uns dos outros para que nenhum interfira com o que outro está a fazer. Quando qualquer uma destas três capacidades falha, a abstração deixa de se sustentar, e o comportamento do sistema torna-se difícil de explicar apenas a partir da interface que ele expõe.

A virtualização não é uma técnica recente

- Os mainframes IBM System/360, na década de 1960, já particionavam um único computador entre múltiplos sistemas operativos monoutilizador – o antecedente direto da máquina virtual atual.
- O interesse esmoreceu nos anos 1980 e 1990, quando os sistemas operativos multiutilizador tornaram redundante grande parte dessa necessidade.
- Foi retomada a partir de 1998, com a VMware, e consolidada entre 2005 e 2008 com suporte de hardware dedicado (Intel VT-x, AMD-V) e hypervisors como Xen e KVM.
- É esta maturação de várias décadas – não uma inovação recente – que sustenta o modelo de *cloud* elástica discutido na Parte 4.

É tentador pensar na virtualização como uma invenção da era da cloud, mas a história conta outra coisa. Os mainframes IBM System/360, já na década de 1960, particionavam um único computador físico entre vários sistemas operativos monoutilizador, o antecedente direto daquilo que hoje chamamos máquina virtual. O interesse esmoreceu nos anos oitenta e noventa, quando os sistemas operativos multiutilizador tornaram redundante grande parte dessa necessidade original, cada utilizador já conseguia partilhar uma única máquina sem precisar de uma máquina virtual inteira só para si. A ideia foi retomada a partir de 1998 com a fundação da VMware, e consolidou-se definitivamente entre 2005 e 2008, quando surgiu suporte de hardware dedicado, a Intel com VT-x e a AMD com AMD-V, e hypervisors como Xen e KVM amadureceram o suficiente para uso generalizado. É esta maturação ao longo de várias décadas, e não uma inovação recente, que sustenta o modelo de cloud elástica que vamos discutir na Parte 4 desta mesma aula.

A definição formal de Popek e Goldberg

- Referência clássica: Gerald J. Popek e Robert P. Goldberg, *Formal Requirements for Virtualizable Third Generation Architectures*, Communications of the ACM, 1974.
- Os autores formalizam a exigência de isolamento enunciada atrás: um *Virtual Machine Monitor* pode ser construído sempre que o conjunto de instruções sensíveis de uma arquitetura for subconjunto do conjunto de instruções privilegiadas.
- Quando essa condição falha – como sucedia em arquiteturas x86 anteriores a 2005 –, algumas instruções sensíveis não geram exceção, e o isolamento tem de ser assegurado por outros meios.
- O resultado é uma condição verificável sobre uma arquitetura de processador, e não uma propriedade de um produto concreto.

A referência clássica sobre este assunto é um artigo de 1974, de Gerald J. Popek e Robert P. Goldberg, publicado na Communications of the ACM com o título *Formal Requirements for Virtualizable Third Generation Architectures*. Os autores formalizaram matematicamente a exigência de isolamento já discutida informalmente atrás: um *Virtual Machine Monitor* pode ser construído sempre que o conjunto de instruções sensíveis de uma arquitetura de processador for um subconjunto do conjunto de instruções privilegiadas dessa mesma arquitetura. Quando essa condição não se verifica, como acontecia nas arquiteturas x86 anteriores a 2005, algumas instruções sensíveis não geram exceção ao serem executadas em modo não privilegiado, e o isolamento tem de ser garantido por outros meios, mais complexos e mais dispendiosos de implementar. O resultado interessante desta formalização é que se trata de uma condição verificável sobre uma arquitetura de processador enquanto tal, e não de uma propriedade de um produto de software concreto.

Hypervisor: duas arquiteturas

- **Hypervisor:** o componente que implementa a multiplexagem de hardware entre máquinas virtuais.
- **Tipo 1 (nativo)** – executa diretamente sobre o hardware, sem sistema operativo hospedeiro. Exemplos: Xen, KVM, VMware ESXi.
- **Tipo 2 (alojado)** – executa como aplicação sobre um sistema operativo hospedeiro, que continua a mediar o acesso ao hardware. Exemplo: VirtualBox.
- A diferença é, também ela, uma questão de indirecção: o tipo 2 introduz uma camada adicional – o sistema operativo hospedeiro – entre o hypervisor e o hardware.

O hypervisor é o componente de software que implementa, na prática, a multiplexagem de hardware entre várias máquinas virtuais. Existem duas arquiteturas fundamentalmente diferentes. Um hypervisor de tipo 1, dito nativo, executa diretamente sobre o hardware, sem qualquer sistema operativo hospedeiro pelo meio, como acontece com o Xen, o KVM ou o VMware ESXi. Um hypervisor de tipo 2, dito alojado, executa como uma aplicação normal sobre um sistema operativo hospedeiro, que continua a mediar todo o acesso ao hardware físico, como é o caso do VirtualBox. A diferença entre os dois é, também ela, uma questão de indireção, tema que vamos aprofundar já a seguir: o tipo 2 introduz uma camada adicional, o sistema operativo hospedeiro, entre o hypervisor e o hardware que ele pretende controlar.

Duas arquiteturas de hypervisor



O diagrama mostra lado a lado as duas arquiteturas descritas no slide anterior. No hypervisor de tipo 1, o caminho entre a máquina virtual e o hardware físico passa por uma única camada. No hypervisor de tipo 2, esse mesmo caminho passa por duas camadas, o hypervisor e, acima dele, o sistema operativo hospedeiro. Esta diferença de profundidade explica diretamente por que motivo os hypervisors de tipo 1 são a escolha quase universal em ambientes de produção, onde cada camada adicional representa desempenho perdido, enquanto os de tipo 2 continuam populares em portáteis e postos de trabalho, onde a conveniência de correr dentro de um sistema operativo já instalado pesa mais do que o desempenho máximo alcançável.

Acrescentar um nível de indirecção

- Introduzir um nível de indirecção significa deixar de referir um recurso diretamente e passar a referi-lo através de um intermediário que faz a tradução.
- É o que permite substituir, mover ou replicar o recurso sem alterar quem o refere.
- Praticamente todas as propriedades que valorizamos – portabilidade, elasticidade, tolerância a falhas – decorrem da existência dessa camada intermédia.
- Formulação clássica, atribuída a David Wheeler: qualquer problema em informática pode ser resolvido acrescentando um nível de indirecção.

Introduzir um nível de indirecção significa, em termos simples, deixar de referir um recurso diretamente e passar a referi-lo através de um intermediário que se encarrega de traduzir essa referência para o recurso real. É exatamente o que um endereço de memória virtual faz em relação a um endereço físico, ou o que um nome de domínio faz em relação a um endereço IP. O valor prático desta ideia é enorme: é o que permite substituir, mover ou replicar o recurso real sem que quem o refere precise sequer de saber que isso aconteceu. Praticamente todas as propriedades que a engenharia de infraestrutura mais valoriza, portabilidade, elasticidade, tolerância a falhas, decorrem, direta ou indiretamente, da existência de uma camada intermédia deste tipo. Há uma formulação clássica desta ideia, atribuída a David Wheeler, que diz que qualquer problema em informática pode ser resolvido acrescentando um nível de indirecção.

O que a indirecção custa

- A mesma formulação costuma ser completada com uma ressalva: o problema que a indirecção não resolve é o de haver indirecção a mais.
- Cada camada acrescenta latência, consome recursos e introduz um novo modo de falha.
- Acrescenta também **opacidade**: o diagnóstico passa a exigir atravessar todas as camadas.
- Decisão de engenharia recorrente nesta UC: *que indirecção compensa introduzir, e a que custo.*
- Não existe resposta universal – depende dos requisitos de cada sistema.

A frase de David Wheeler costuma vir acompanhada de uma segunda parte, menos citada mas igualmente importante: o único problema que a indireção não resolve é o de haver indireção a mais. Cada camada intermédia que se acrescenta a um sistema tem um custo real. Acrescenta latência, porque cada tradução consome tempo. Consome recursos próprios, porque a camada em si tem de correr nalgum lado. E introduz um novo modo de falha, porque agora há mais um componente que pode avariar. Introduz também algo mais difícil de quantificar, a opacidade: diagnosticar um problema passa a exigir atravessar todas as camadas envolvidas, uma a uma, em vez de olhar diretamente para o recurso final. Esta tensão vai aparecer repetidamente ao longo do semestre como uma decisão de engenharia concreta: que indireção compensa introduzir, e a que custo. Não existe uma resposta universal, a resposta depende sempre dos requisitos específicos do sistema em causa.

Parte 2

Princípios de arquitetura

A segunda parte da aula desloca a atenção da virtualização enquanto mecanismo concreto para um conjunto de princípios de arquitetura mais gerais, que atravessam praticamente toda a engenharia de sistemas distribuídos e que vão reaparecer, sem aviso prévio, em quase todos os blocos seguintes do programa: abstrações, separação entre mecanismo e política, idempotência e controlo automático.

Onde as abstrações falham

- Uma abstração é um contrato: expõe um conjunto de operações e oculta a forma como são realizadas.
- O seu valor está no que oculta; a sua fragilidade está exatamente no mesmo sítio.
- **Abstrações com fugas:** propriedades da implementação que se tornam observáveis através da interface – tipicamente o desempenho.
- Um disco virtual comporta-se como um disco até o armazenamento subjacente saturar. A interface não mudou; o comportamento sim.
- Consequência prática: o administrador de infraestrutura tem de compreender *pelo menos uma camada abaixo* daquela em que opera.

Uma abstração pode ser entendida como um contrato: expõe um conjunto limitado de operações e esconde deliberadamente a forma como essas operações são realmente executadas por baixo. O valor de uma abstração está precisamente no que ela oculta, é isso que permite a quem a usa não ter de pensar em todos os detalhes de implementação. Mas a sua fragilidade está exatamente no mesmo sítio. Fala-se em abstrações com fugas quando propriedades da implementação, que deveriam estar escondidas, se tornam de qualquer forma observáveis através da interface, tipicamente sob a forma de desempenho anómalo. Um disco virtual comporta-se exatamente como um disco físico até o armazenamento subjacente saturar, e nesse momento a interface não mudou nada, mas o comportamento mudou drasticamente. A consequência prática desta observação é que um administrador de infraestrutura competente precisa de compreender, pelo menos, uma camada abaixo daquela em que está a trabalhar diretamente.

Mecanismo e política

- **Mecanismo:** o que o sistema é *capaz* de fazer. **Política:** o que o sistema *deve* fazer em cada circunstância.
- Separá-los permite alterar a política sem reimplementar o sistema.
- Exemplo estrutural: um orquestrador fornece o mecanismo de colocar cargas de trabalho em nós; a política de colocação é declarada à parte, pelo operador.
- É este princípio que torna uma plataforma *configurável* em vez de meramente *programável*.
- Reaparecerá em praticamente todos os blocos do programa.

Esta é outra distinção que reaparece constantemente em engenharia de sistemas. O mecanismo é aquilo que o sistema é capaz de fazer, a capacidade técnica disponível. A política é aquilo que o sistema deve fazer em cada circunstância concreta, a decisão sobre como usar essa capacidade. Separar os dois permite alterar a política sem ter de reimplementar todo o sistema. Um exemplo estrutural, que vamos encontrar de novo já na Aula 05: um orquestrador de containers fornece o mecanismo de colocar cargas de trabalho em nós de um cluster, mas a política de colocação, que carga vai para que nó e em que condições, é declarada à parte, pelo operador do sistema, sem tocar no código do orquestrador. É precisamente este princípio de separação que torna uma plataforma configurável em vez de meramente programável, e vai reaparecer, sob roupagens diferentes, em praticamente todos os blocos do programa.

O argumento end-to-end

- Já vem do contexto do desenho de sistemas distribuídos.
- Uma função só pode ser garantida de forma completa e correta com conhecimento que reside nos *extremos* da comunicação.
- Implementá-la em camadas intermédias é, no limite, redundante – ainda que possa justificar-se por razões de desempenho.
- Consequência para infraestrutura: a camada de infraestrutura não deve tentar garantir propriedades que só a aplicação pode assegurar.
- Delimita o que é razoável exigir de uma plataforma, e o que tem de permanecer responsabilidade de quem a usa.

Este argumento nasceu no contexto do desenho de sistemas distribuídos e tem uma formulação relativamente simples de enunciar, ainda que profunda nas suas implicações. Uma função só pode ser garantida de forma completa e correta se implementada com conhecimento que reside nos extremos da comunicação, ou seja, nas aplicações que efetivamente enviam e recebem os dados. Implementar essa mesma função em camadas intermédias da rede ou da infraestrutura é, no limite, redundante, ainda que por vezes se justifique fazê-lo por razões de desempenho, para detetar erros mais cedo por exemplo. A consequência prática para infraestrutura é direta: a camada de infraestrutura não deve tentar garantir propriedades que, no fundo, só a própria aplicação está em posição de assegurar corretamente. Este argumento delimita aquilo que é razoável exigir de uma plataforma de infraestrutura, e aquilo que tem de permanecer responsabilidade de quem constrói a aplicação que corre por cima dela.

Declarativo vs imperativo

- **Imperativa:** descreve-se a *sequência de ações* a executar. O resultado depende do estado inicial.
- **Declarativa:** descreve-se o *estado final pretendido*. Cabe ao sistema determinar as ações necessárias.
- A forma declarativa é composicional e verificável: pode comparar-se a especificação com a realidade observada.
- A forma imperativa não o permite – um conjunto de comandos não se compara com um estado.
- Esta distinção é o fundamento conceptual da automatização moderna de infraestrutura.

Numa abordagem imperativa descreve-se a sequência exata de ações que devem ser executadas, passo a passo. O problema desta abordagem é que o resultado final depende do estado inicial da máquina, e esse estado inicial nem sempre é conhecido com certeza. Numa abordagem declarativa descreve-se, em vez disso, o estado final que se pretende atingir, e cabe ao próprio sistema determinar que ações são necessárias para lá chegar a partir de onde quer que esteja. A forma declarativa tem uma vantagem estrutural importante: é composicional e verificável, porque é sempre possível comparar a especificação declarada com o estado real observado no sistema, e ver se coincidem. A forma imperativa não permite esta comparação, um conjunto de comandos já executados não se compara diretamente com um estado presente. Esta distinção, que pode parecer académica à primeira vista, é o fundamento conceptual de toda a automatização moderna de infraestrutura, que vamos estudar em detalhe na Aula 07.

Idempotência e convergência

- Uma operação é **idempotente** se o efeito de a aplicar n vezes é indistinguível do de a aplicar uma vez.
- Formalmente: $f(f(x)) = f(x)$ para todo o x do domínio.
- A idempotência é o que permite *repetir com segurança* – e repetir com segurança é o que permite recuperar de falhas parciais sem raciocinar sobre o ponto exato em que a execução foi interrompida.
- **Convergência**: propriedade de um sistema que, aplicando repetidamente a mesma especificação, tende para o estado declarado.
- Sem idempotência, a repetição é perigosa; sem repetição, não há convergência.

Uma operação diz-se idempotente quando o efeito de a aplicar n vezes seguidas é indistinguível do efeito de a aplicar apenas uma vez. Formalmente, $f(f(x)) = f(x)$ para todo o x do domínio da função. A razão por que a idempotência importa tanto em infraestrutura é simples de enunciar mas tem consequências profundas: é o que permite repetir uma operação com segurança, e repetir com segurança é exatamente o que permite recuperar de falhas parciais sem ter de raciocinar sobre o ponto exato em que a execução anterior foi interrompida. Basta correr tudo de novo. A convergência é a propriedade de um sistema que, ao aplicar repetidamente a mesma especificação, tende sempre para o estado que foi declarado, seja qual for o ponto de partida. A relação entre os dois conceitos é direta: sem idempotência, repetir uma operação é perigoso, porque pode agravar o problema em vez de o resolver; e sem a possibilidade de repetir com segurança, não existe convergência possível.

Idempotência na prática

- Suponha um procedimento que acrescenta um servidor a um ficheiro de configuração de um balanceador de carga.
- **Versão não idempotente:** «acrescentar a linha `server 10.0.0.7` ao ficheiro». Correr duas vezes deixa a linha duplicada; o balanceador pode recusar arrancar.
- **Versão idempotente:** «garantir que o ficheiro contém exatamente estes três servidores». Correr as vezes que forem necessárias produz sempre o mesmo ficheiro.
- A diferença aparece quando a execução falha a meio: com a primeira versão é preciso descobrir onde parou; com a segunda, basta repetir.
- É por isso que as ferramentas de automatização que veremos em C3 descrevem *estado*, e não sequências de comandos.

Vale a pena tornar este conceito concreto com um exemplo simples. Suponha um procedimento que acrescenta um servidor a um ficheiro de configuração de um balanceador de carga. Numa versão não idempotente, o procedimento diz apenas acrescentar a linha `server 10.0.0.7` ao ficheiro. Correr este procedimento duas vezes deixa essa linha duplicada no ficheiro, e o balanceador de carga pode muito bem recusar-se a arrancar com uma configuração inválida. Numa versão idempotente, o procedimento diz antes garantir que o ficheiro contém exatamente estes três servidores, nem mais nem menos. Correr este segundo procedimento as vezes que forem necessárias produz sempre exatamente o mesmo ficheiro final, seja qual for o estado em que o ficheiro estava antes. A diferença entre as duas versões torna-se decisiva precisamente quando a execução falha a meio do caminho: com a primeira versão é preciso investigar onde é que a execução parou antes de decidir o que fazer a seguir; com a segunda versão, basta correr tudo outra vez. É por esta razão que as ferramentas de automatização que vamos ver no bloco C3 do programa descrevem sempre estado desejado, e nunca sequências de comandos a executar.

Controlo em malha fechada

- Vários sistemas que estudaremos são, estruturalmente, **controladores em malha fechada**.
- Elementos: um *valor de referência* (estado desejado), uma *medição* (estado observado), um *erro* (a diferença) e uma *ação* que tende a reduzi-lo.
- A execução é contínua, não episódica: o sistema não «faz» e termina – observa e corrige indefinidamente.
- Daqui decorre uma propriedade notável: a recuperação de falhas não precisa de ser programada como caso especial. É o comportamento normal do ciclo.

Muitos dos sistemas que vamos estudar ao longo do semestre são, estruturalmente, controladores em malha fechada, um conceito que vem da engenharia de controlo clássica mas que se aplica quase sem alterações a software de infraestrutura. Um controlador deste tipo tem quatro elementos: um valor de referência, que é o estado desejado; uma medição, que é o estado atualmente observado no sistema real; um erro, que é simplesmente a diferença entre os dois; e uma ação, que tenta reduzir esse erro. A execução deste ciclo é contínua, não episódica, o sistema não faz uma coisa e termina, observa e corrige indefinidamente, sem parar. Desta continuidade decorre uma propriedade particularmente elegante: a recuperação de falhas não precisa de ser programada como um caso especial à parte, tratado de forma diferente do resto. É apenas o comportamento normal do ciclo a funcionar como sempre funcionou.

Como funciona o ciclo



O ciclo é contínuo, não episódico.
A recuperação de falhas não é um caso especial — é o comportamento normal do ciclo.

O diagrama mostra os quatro elementos do controlador em malha fechada descrito no slide anterior, ligados num circuito que se repete sem fim: o estado desejado é comparado com o estado observado, a diferença entre ambos gera uma ação corretiva, essa ação altera o sistema real, o sistema real é medido de novo, e o ciclo recomeça. Guarde bem esta imagem, porque é exatamente este desenho que está por trás do funcionamento do Kubernetes, que vamos estudar em profundidade na Aula 05, e de praticamente todas as ferramentas modernas de automatização de infraestrutura.

Acoplamento entre componentes

- **Acoplamento:** medida da dependência entre componentes – quanto é que a alteração ou falha de um afeta os outros.
- Distinguem-se várias dimensões: temporal (têm de estar ativos em simultâneo?), de localização, de formato de dados e de implementação.
- Reduzir acoplamento aumenta a capacidade de evoluir e de falhar isoladamente; aumenta também o número de componentes a gerir.
- Não existe um nível de acoplamento universalmente correto – existe um nível adequado a um dado conjunto de requisitos.

O acoplamento mede a dependência entre dois componentes de um sistema: quanto é que a alteração, ou a falha, de um deles afeta o funcionamento do outro. Costuma ser útil distinguir várias dimensões distintas de acoplamento. O acoplamento temporal pergunta se os dois componentes têm de estar ativos ao mesmo tempo para comunicarem. O acoplamento de localização pergunta se um componente precisa de saber onde o outro está fisicamente. O acoplamento de formato de dados pergunta se ambos têm de concordar exatamente na estrutura da informação trocada. E o acoplamento de implementação pergunta se um componente depende de detalhes internos de como o outro foi construído. Reduzir acoplamento entre componentes aumenta a capacidade do sistema de evoluir ao longo do tempo e de falhar de forma isolada, sem arrastar consigo tudo o resto, mas tem um preço direto: aumenta também o número de componentes distintos que é preciso gerir separadamente. Não existe um nível de acoplamento universalmente correto, existe apenas um nível adequado a um conjunto específico de requisitos.

Parte 3

O argumento económico

A terceira parte da aula muda de registo e passa a olhar para a infraestrutura do ponto de vista económico. Porque é que consolidar recursos numa infraestrutura partilhada compensa financeiramente, e em que condições é que essa vantagem desaparece, é o fio condutor destas páginas.

Porque se consolida

- Um sistema dimensionado para o seu pico de procura permanece subutilizado durante a maior parte do tempo.
- Dimensionar para a média torna-o incapaz de responder ao pico.
- O dilema é insolúvel *para um sistema isolado*. Deixa de o ser quando se agregam vários sistemas independentes.
- A consolidação não é apenas uma conveniência operacional: assenta numa propriedade estatística da agregação de procuras independentes.

Considere um sistema dimensionado exatamente para o seu pico de procura previsto. Na maior parte do tempo, fora desse pico, esse sistema fica subutilizado, e todo o investimento feito na capacidade extra está parado sem produzir valor. Considere agora o extremo oposto, um sistema dimensionado apenas para a procura média: torna-se incapaz de responder adequadamente sempre que a procura sobe acima dessa média. Para um sistema isolado, sozinho, este dilema não tem solução satisfatória, é preciso escolher entre desperdício e insuficiência. O dilema deixa de ser insolúvel, contudo, quando se agregam vários sistemas independentes num só. A consolidação de infraestrutura não é apenas uma conveniência de gestão que facilita a vida a quem administra os sistemas, assenta numa propriedade estatística real da agregação de procuras independentes, que o próximo slide desenvolve com mais detalhe.

Quando a consolidação não compensa

- Agregar muitas procuras independentes torna a procura total proporcionalmente *mais previsível* do que cada uma isoladamente.
- Daí resulta que a margem de segurança necessária, em proporção, diminui à medida que se agregam cargas – é esta a base da economia de escala.
- A hipótese crítica é a **independência**. Se as cargas forem correlacionadas, por exemplo com pico à mesma hora, a previsibilidade desaparece e o ganho com ela.
- Um modelo é útil sobretudo por ser explícito quanto às condições em que deixa de se aplicar.

Agregar muitas procuras independentes torna a procura total, olhada como um todo, proporcionalmente mais previsível do que cada procura individual olhada isoladamente. Isto é uma consequência direta de como variam somas de variáveis aleatórias independentes. Da agregação decorre que a margem de segurança necessária, em proporção ao total, diminui à medida que se agregam mais cargas de trabalho distintas, e é exatamente esta a base matemática por trás da economia de escala de que tanto se fala em cloud computing. Mas há uma hipótese crítica escondida neste raciocínio: a independência das cargas. Se as cargas de trabalho forem correlacionadas entre si, por exemplo se todas tiverem o seu pico exatamente à mesma hora do dia, a previsibilidade que a agregação prometia desaparece, e o ganho económico desaparece com ela. Um modelo teórico é útil sobretudo por deixar explícitas as condições em que deixa de se aplicar, e este é um bom exemplo disso mesmo.

Utilização e latência

- Uma intuição comum sugere que se deve operar a infraestrutura com utilização próxima de 100%, para não desperdiçar.
- A teoria de filas de espera mostra que essa intuição está errada.
- Para um sistema elementar com chegadas e serviço exponenciais (modelo M/M/1), o tempo médio de resposta R relaciona-se com o tempo de serviço S e a utilização ρ por

$$R = \frac{S}{1 - \rho}$$

- À medida que $\rho \rightarrow 1$, o tempo de resposta cresce sem limite.

Existe uma intuição muito comum, e enganadora, que sugere que se deve operar a infraestrutura o mais próximo possível de cem por cento de utilização, para não haver qualquer desperdício de capacidade paga. A teoria de filas de espera mostra, com bastante clareza, que essa intuição está errada. Para um sistema elementar de filas com chegadas e tempos de serviço exponenciais, conhecido na literatura como modelo M/M/1, o tempo médio de resposta R relaciona-se com o tempo de serviço S e a utilização ρ através da fórmula

$$R = \frac{S}{1 - \rho}.$$

À medida que $\rho \rightarrow 1$, ou seja, à medida que a utilização se aproxima dos cem por cento, o tempo de resposta cresce sem qualquer limite matemático. Este resultado, simples de escrever, tem consequências práticas enormes para quem dimensiona infraestrutura.

O que isto significa na prática

- Avaliando R/S para valores crescentes de utilização:
 - $\rho = 0,50 \Rightarrow R = 2S$
 - $\rho = 0,80 \Rightarrow R = 5S$
 - $\rho = 0,90 \Rightarrow R = 10S$
 - $\rho = 0,99 \Rightarrow R = 100S$
- A degradação não é linear.
- Operar com folga não é desperdício – é o preço da previsibilidade da latência.

Vale a pena traduzir a fórmula anterior em números concretos, avaliando a razão R/S para valores crescentes de utilização. A cinquenta por cento de utilização, o tempo de resposta é o dobro do tempo de serviço. A oitenta por cento, já é cinco vezes maior. A noventa por cento, dez vezes maior. E a noventa e nove por cento de utilização, o tempo de resposta chega a ser cem vezes o tempo de serviço original. Repare como a degradação não é, de forma alguma, linear com a utilização, é uma curva que se torna cada vez mais íngreme à medida que se aproxima do limite máximo. A conclusão prática é contraintuitiva mas inevitável: operar com alguma folga de capacidade não é desperdício, é literalmente o preço que se paga pela previsibilidade da latência que os utilizadores do sistema vão sentir.

O que torna a elasticidade útil

- **Elasticidade:** capacidade de ajustar a capacidade provisionada à procura observada, em ambos os sentidos.
- O seu valor depende de três grandezas: a amplitude da variação da procura, o tempo de reação do mecanismo de ajuste e a granularidade da unidade faturada.
- Se o tempo de reação for superior à duração do pico, a elasticidade não produz benefício.

A elasticidade é a capacidade de ajustar a capacidade provisionada à procura efetivamente observada, em ambos os sentidos, tanto para cima como para baixo. O valor real que a elasticidade traz depende de três grandezas distintas: a amplitude da variação da procura ao longo do tempo, o tempo de reação do mecanismo responsável por esse ajuste, e a granularidade da unidade que é efetivamente faturada ao cliente. Há uma condição simples que decide se a elasticidade compensa ou não num dado cenário: se o tempo de reação do mecanismo de ajuste for superior à duração do próprio pico de procura, a elasticidade não chega a produzir qualquer benefício prático, porque quando o sistema finalmente reage, o pico já passou.

Parte 4

O que é, afinal, *cloud*

A quarta parte da aula pergunta, com todo o rigor que a pergunta merece, o que é, afinal, cloud computing. É uma pergunta que parece óbvia até se tentar respondê-la com precisão suficiente para sustentar uma decisão de engenharia real.

Porque precisamos de uma definição

- O termo *cloud* é usado com sentidos muito diferentes consoante o contexto – comercial, técnico ou contratual.
- Num contexto de engenharia, um termo sem fronteiras definidas impede raciocínio rigoroso e comparação entre alternativas.
- A definição do NIST (Mell e Grance, 2011) é a referência habitual porque é *operacional*: enuncia critérios verificáveis.
- Vamos analisá-la como se analisa qualquer definição: identificando o que inclui, o que exclui e o que deixa em aberto.

O termo cloud é usado, no dia a dia, com sentidos bastante diferentes consoante o contexto em que aparece, comercial, técnico ou contratual, e frequentemente sem que quem o usa tenha noção dessa ambiguidade. Num contexto de engenharia, contudo, um termo sem fronteiras bem definidas impede qualquer raciocínio rigoroso e qualquer comparação séria entre alternativas concretas. A definição do NIST, publicada por Mell e Grance em 2011, tornou-se a referência habitual precisamente por ser operacional: em vez de descrever cloud de forma vaga ou aspiracional, enuncia critérios que podem ser verificados um a um. Vamos analisá-la exatamente como se analisa qualquer definição técnica rigorosa: identificando o que ela inclui explicitamente, o que exclui deliberadamente, e o que deixa em aberto para outras discussões.

As cinco características do NIST

- **Self-service sob procura** – o consumidor aprovisiona sem intervenção humana do fornecedor.
- **Acesso alargado à rede** – disponível através de mecanismos padronizados.
- **Partilha de recursos** (*resource pooling*) – recursos servem múltiplos consumidores, com atribuição dinâmica.
- **Elasticidade rápida** – capacidade ajustável, em ambos os sentidos, com rapidez.
- **Serviço medido** – o consumo é monitorizado e reportado de forma transparente.

A definição do NIST assenta em cinco características que, em conjunto, distinguem cloud computing de qualquer outra forma de infraestrutura. Self-service sob procura significa que o consumidor consegue aprovisionar recursos sozinho, sem precisar de qualquer intervenção humana do lado do fornecedor. Acesso alargado à rede significa que os recursos estão disponíveis através de mecanismos de acesso padronizados, não proprietários. Partilha de recursos, ou *resource pooling*, significa que os mesmos recursos físicos servem múltiplos consumidores diferentes, com atribuição dinâmica consoante a procura de cada um. Elasticidade rápida significa que a capacidade aprovisionada pode ser ajustada, em ambos os sentidos, com grande rapidez. E serviço medido significa que o consumo de cada consumidor é monitorizado e reportado de forma transparente, tipicamente para efeitos de faturação.

O que a definição realmente diz

- As cinco não são independentes: o *serviço medido* é pré-condição do *self-service* com responsabilização, e a *partilha de recursos* é pré-condição da *elasticidade*.
- Duas são de natureza técnica (*partilha*, *elasticidade*); três são de natureza *organizacional* – descrevem a relação entre fornecedor e consumidor.
- Consequência frequentemente ignorada: uma infraestrutura pode ser tecnicamente sofisticada e ainda assim não constituir uma *cloud*, se o aprovisionamento exigir intervenção humana.
- A definição é, no essencial, sobre **modo de operação**, não sobre tecnologia empregue.

Estas cinco características não são independentes entre si, ao contrário do que a lista pode sugerir à primeira leitura. O serviço medido é, na prática, uma pré-condição para que o self-service com responsabilização faça sentido, sem medir o consumo, ninguém consegue responsabilizar-se por ele. E a partilha de recursos é, por sua vez, uma pré-condição para que a elasticidade rápida seja sequer possível, só se consegue crescer depressa se houver um conjunto maior de recursos partilhados de onde retirar essa capacidade extra. Vale ainda notar que duas destas características são de natureza técnica, a partilha e a elasticidade, mas as outras três são, na sua essência, de natureza organizacional, descrevem a relação entre fornecedor e consumidor e não propriamente a tecnologia usada por baixo. Uma consequência desta observação, frequentemente ignorada na prática, é que uma infraestrutura pode ser tecnicamente muito sofisticada e ainda assim não constituir uma cloud no sentido desta definição, se o aprovisionamento continuar a exigir intervenção humana de cada vez que é pedido. A definição do NIST é, no essencial, sobre um modo de operação, não sobre que tecnologia concreta está a ser empregue.

O que significam IaaS, PaaS e SaaS

- **IaaS** — *Infrastructure as a Service*

O fornecedor entrega recursos brutos: capacidade de computação, armazenamento e rede.

O cliente instala e gere o sistema operativo e tudo o que corre acima dele.

- **PaaS** — *Platform as a Service*

O fornecedor entrega um ambiente de execução já pronto. O cliente entrega a aplicação e

os dados, sem gerir o sistema operativo.

- **SaaS** — *Software as a Service*

O fornecedor entrega a aplicação a funcionar. Ao cliente resta configurar acessos e gerir os seus dados.

Estes três acrónimos descrevem três formas diferentes de repartir responsabilidade entre fornecedor e cliente. Em Infrastructure as a Service, o fornecedor entrega apenas recursos brutos, capacidade de computação, armazenamento e rede, e o cliente instala e gere sozinho o sistema operativo e tudo o que corre por cima dele. Em Platform as a Service, o fornecedor entrega já um ambiente de execução completo e pronto a usar, e o cliente limita-se a entregar a sua aplicação e os seus dados, sem nunca ter de gerir diretamente o sistema operativo subjacente. Em Software as a Service, o fornecedor entrega a própria aplicação já a funcionar, e ao cliente resta apenas configurar acessos de utilizadores e gerir os seus próprios dados dentro dessa aplicação.

IaaS, PaaS e SaaS

- **IaaS, PaaS e SaaS** são habitualmente apresentados como níveis de abstração crescente.
- É mais produtivo lê-los como diferentes **repartições de responsabilidade** sobre uma mesma pilha de camadas.
- Em cada modelo desloca-se a fronteira entre o que o fornecedor gere e o que o consumidor gere.
- Cada deslocação troca *controlo* por *redução de encargo operacional* – e é essa troca, não o nível de abstração em si, que fundamenta a escolha.

Estes três modelos costumam ser apresentados como níveis crescentes de abstração, como se fossem simplesmente três patamares de uma escada. É mais produtivo, contudo, lê-los como diferentes repartições de responsabilidade sobre uma mesma pilha de camadas de software e hardware. Em cada um dos três modelos desloca-se a fronteira entre aquilo que o fornecedor gere por si e aquilo que o consumidor continua a ter de gerir. E cada deslocação desta fronteira troca controlo direto por redução do encargo operacional do lado do cliente, é essa troca concreta, e não o nível de abstração em si mesmo considerado, que acaba por fundamentar a escolha entre um modelo e outro num caso real.

Onde fica a fronteira



O diagrama mostra exatamente onde fica essa fronteira em cada um dos três modelos, camada a camada, desde o hardware físico até à aplicação final que o utilizador vê. Repare como a área que o fornecedor gere cresce de IaaS para PaaS e de PaaS para SaaS, enquanto a área que resta ao cliente gerir encolhe na mesma proporção. Esta unidade curricular, como veremos ao longo do semestre, situa-se de forma bastante consistente ao nível de IaaS, a camada em que a infraestrutura ainda é explicitamente construída e administrada por quem a usa, em vez de já vir pronta.

O que a fronteira determina

- A fronteira determina simultaneamente a responsabilidade *operacional* e a responsabilidade *de segurança*.
- Uma fonte recorrente de incidentes é a suposição de que o que está acima da fronteira é assegurado por quem está abaixo dela.
- A fronteira determina também o **grau de dependência do fornecedor**: quanto mais alto o modelo, mais específicas as interfaces utilizadas, e mais dispendiosa a migração.
- Esta unidade curricular situa-se predominantemente ao nível de **IaaS** – a camada onde a infraestrutura é explicitamente construída e administrada.

Esta fronteira entre fornecedor e cliente determina, em simultâneo, duas coisas distintas: a responsabilidade operacional do dia a dia, e a responsabilidade de segurança do sistema. Uma fonte de incidentes de segurança bastante recorrente na indústria é precisamente a suposição errada de que tudo o que está acima da fronteira é automaticamente assegurado por quem está abaixo dela, quando muitas vezes não é esse o caso, e o próprio contrato de serviço costuma deixar isso explícito. A fronteira determina também o grau de dependência que o cliente passa a ter do seu fornecedor: quanto mais alto se está no modelo, IaaS, PaaS ou SaaS, mais específicas costumam ser as interfaces e as ferramentas usadas, e mais dispendiosa se torna, no futuro, uma eventual migração para outro fornecedor. Esta própria unidade curricular situa-se predominantemente ao nível de IaaS, exatamente a camada em que a infraestrutura ainda é explicitamente construída e administrada, em vez de vir já pronta a usar.

Pública, privada, híbrida

- **Pública, privada, híbrida e comunitária** distinguem-se pelo *locus de controlo* e pelo conjunto de consumidores servidos.
- A distinção não é primariamente técnica: as mesmas tecnologias podem sustentar qualquer um dos modelos.
- Os critérios de escolha são tipicamente de natureza regulatória, contratual ou de governação – por exemplo, requisitos de localização dos dados.
- O modelo híbrido introduz uma dificuldade específica: a necessidade de manter coerência operacional entre ambientes com modelos de gestão distintos.

Pública, privada, híbrida e comunitária são quatro formas de classificar uma cloud, mas a distinção entre elas assenta num critério diferente do que distinguia IaaS de PaaS e de SaaS: aqui o que importa é o locus de controlo e o conjunto de consumidores que a infraestrutura serve. Esta distinção não é, ao contrário do que se poderia pensar, primariamente uma questão técnica, as mesmas tecnologias concretas podem perfeitamente sustentar qualquer um dos quatro modelos. Os critérios que normalmente decidem a escolha entre eles são de natureza regulatória, contratual ou de governação interna da organização, por exemplo requisitos legais sobre onde os dados têm fisicamente de residir. O modelo híbrido introduz uma dificuldade específica, que vale a pena assinalar: a necessidade de manter coerência operacional entre ambientes que, por definição, têm modelos de gestão distintos entre si.

O que a definição deixa de fora

- Nada diz sobre **desempenho**: um sistema lento pode satisfazer integralmente as cinco características.
- Nada diz sobre **dependabilidade**: a definição é ortogonal à fiabilidade do serviço.
- Nada diz sobre **custo**: a elasticidade não implica economia.
- Estas dimensões, ausentes da definição, são precisamente aquelas em que reside o trabalho de engenharia – e o objeto das partes seguintes desta aula.

É tão instrutivo perceber o que a definição do NIST não diz como perceber o que ela diz. A definição nada diz sobre desempenho: um sistema lento, mesmo muito lento, pode satisfazer integralmente as cinco características e continuar a chamar-se cloud sem qualquer contradição. Nada diz sobre dependabilidade: a definição é inteiramente ortogonal à fiabilidade real do serviço prestado. E nada diz sobre custo: elasticidade não implica, de forma alguma, economia, uma cloud elástica pode perfeitamente sair mais cara do que a alternativa. Estas três dimensões, ausentes da definição formal, são precisamente aquelas em que reside a maior parte do trabalho real de engenharia de infraestrutura, e são também o objeto direto das partes seguintes desta aula.

Parte 5

Qualidade e dependabilidade

A quinta parte da aula entra no domínio da qualidade e da dependabilidade, e introduz um vocabulário preciso, disponibilidade, fiabilidade, falta, erro, falha, SLI, SLO, SLA, que vai ser usado sem explicação adicional em praticamente todas as aulas seguintes.

Quando um atributo se torna requisito

- Requisitos *funcionais* dizem o que o sistema faz; *atributos de qualidade* dizem com que propriedades o faz.
- Em infraestrutura, são os atributos de qualidade que determinam a arquitetura – a funcionalidade é frequentemente trivial.
- Um atributo só é um requisito quando é **quantificado e verificável**. «O sistema deve ser fiável» não é um requisito: não é possível verificar se foi cumprido.
- Tornar um atributo operacional exige definir a grandeza medida, o método de medição e o valor de referência.

Requisitos funcionais descrevem o que um sistema faz. Atributos de qualidade descrevem com que propriedades esse sistema o faz, quão rápido, quão disponível, quão seguro. Em infraestrutura, são sobretudo os atributos de qualidade que acabam por determinar a arquitetura final de um sistema, porque a funcionalidade em si é, muitas vezes, relativamente trivial de implementar. Mas há uma condição importante para que um atributo de qualidade funcione realmente como um requisito: tem de ser quantificado e verificável. A frase o sistema deve ser fiável não é, tecnicamente, um requisito, porque não há maneira de verificar objetivamente se foi cumprida ou não. Tornar um atributo operacional, no sentido técnico da palavra, exige definir com precisão três coisas: a grandeza exata que vai ser medida, o método usado para a medir, e o valor de referência que se considera aceitável.

Os atributos de dependabilidade

- Quadro de referência estabelecido por Avižienis, Laprie, Randell e Landwehr (2004).
- **Dependabilidade:** capacidade de um sistema merecer confiança justificada no serviço que presta.
- Decompõe-se em atributos mensuráveis:
 - *Disponibilidade* – prontidão para prestar serviço correto.
 - *Fiabilidade* – continuidade do serviço correto ao longo do tempo.
 - *Integridade* – ausência de alterações impróprias do estado.
 - *Manutenibilidade* – aptidão para ser reparado e modificado.

O quadro de referência mais citado nesta área foi estabelecido por Avižienis, Laprie, Randell e Landwehr, num artigo de 2004 que se tornou canónico. Dependabilidade define-se como a capacidade de um sistema merecer confiança justificada no serviço que presta, e decompõe-se em quatro atributos que se podem medir separadamente. Disponibilidade é a prontidão do sistema para prestar um serviço correto num dado instante. Fiabilidade é a continuidade desse serviço correto ao longo de um intervalo de tempo. Integridade é a ausência de alterações impróprias ao estado do sistema. E manutenibilidade é a aptidão do sistema para ser reparado e modificado quando necessário. O próximo slide mostra por que motivo os dois primeiros, disponibilidade e fiabilidade, são frequentemente confundidos, apesar de medirem coisas genuinamente diferentes.

Disponibilidade não é fiabilidade

- **Disponibilidade** é uma proporção de tempo; **fiabilidade** é a probabilidade de funcionamento ininterrupto durante um intervalo.
- Um sistema que falha frequentemente mas recupera em segundos pode ter disponibilidade elevada e fiabilidade baixa.
- Um sistema que falha raramente mas demora dias a recuperar pode ter fiabilidade elevada e disponibilidade baixa.
- A distinção é determinante: os dois atributos exigem investimentos diferentes – respetivamente, em recuperação rápida e em prevenção de falhas.

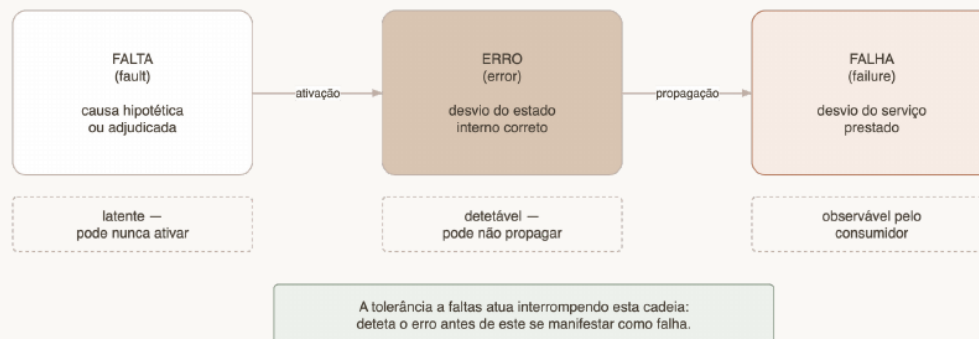
Disponibilidade é, no fundo, uma proporção de tempo, a fração do tempo total em que o sistema esteve a funcionar corretamente. Fiabilidade é a probabilidade de o sistema funcionar sem qualquer interrupção durante um intervalo de tempo especificado à partida. A diferença entre as duas fica clara com dois exemplos opostos. Um sistema que falha com bastante frequência mas que recupera em poucos segundos de cada vez pode facilmente ter disponibilidade elevada e, ao mesmo tempo, fiabilidade baixa. Um sistema que falha raramente mas que, quando falha, demora dias inteiros a recuperar, pode ter fiabilidade elevada e disponibilidade baixa, apesar de falhar muito menos vezes. Esta distinção não é apenas académica, é determinante na prática, porque os dois atributos exigem tipos de investimento completamente diferentes: a disponibilidade beneficia sobretudo de mecanismos de recuperação rápida, enquanto a fiabilidade beneficia sobretudo de prevenção de falhas antes de elas sequer acontecerem.

Falta, erro e falha

- **Falta** (*fault*) – a causa adjudicada ou hipotética de um erro. Pode permanecer latente indefinidamente.
- **Erro** (*error*) – desvio do estado do sistema relativamente ao estado correto. É a manifestação interna da falta.
- **Falha** (*failure*) – o serviço prestado desvia-se do serviço correto. É a manifestação externa do erro.
- A cadeia *não* é determinística: uma falta pode nunca produzir erro, e um erro pode nunca propagar-se até à falha.
- Os mecanismos de tolerância a falhas atuam precisamente interrompendo esta cadeia.

Estes três termos formam uma cadeia causal precisa, que convém não confundir uns com os outros. Uma falta, em inglês *fault*, é a causa adjudicada ou hipotética de um erro, e pode permanecer completamente latente durante muito tempo sem nunca se manifestar, um bug no código que nunca chega a ser executado é uma falta latente. Um erro, em inglês *error*, é o desvio do estado interno do sistema relativamente ao estado que seria correto, é a manifestação interna de uma falta que se ativou. E uma falha, em inglês *failure*, é quando o serviço efetivamente prestado ao exterior se desvia do serviço correto esperado, é a manifestação externa e visível do erro. É essencial perceber que esta cadeia não é determinística em nenhum dos dois sentidos: uma falta pode nunca chegar a produzir um erro, e um erro pode nunca chegar a propagar-se até se tornar uma falha visível ao exterior. Os mecanismos de tolerância a falhas atuam precisamente interrompendo esta cadeia num ponto ou noutro, antes que a falta se transforme em falha visível.

Como uma falta chega a falha



O diagrama traça visualmente o percurso completo que uma falta pode fazer até se manifestar como uma falha visível para o utilizador final, passando pelo estado intermédio de erro. Siga o percurso da esquerda para a direita e repare nos pontos em que a cadeia pode ser interrompida antes de chegar ao fim: é precisamente nesses pontos de interrupção que atuam os mecanismos de deteção, contenção e recuperação que vamos discutir em detalhe nas próximas aulas.

Modelos de falta

- Um sistema tolerante a falhas é-o *sempre relativamente a um modelo de falta declarado*. Não existe tolerância genérica.
- Modelos habituais, por ordem crescente de severidade:
 - **Paragem** (*crash*) – o componente deixa de responder e não volta a agir.
 - **Omissão** – o componente ocasionalmente não responde.
 - **Temporal** – responde, mas fora do intervalo esperado.
 - **Arbitrária** (bizantina) – pode exibir qualquer comportamento, incluindo respostas incorretas e inconsistentes.
- Quanto mais severo o modelo tolerado, mais dispendioso o mecanismo exigido.

Um sistema tolerante a falhas é sempre tolerante a falhas relativamente a um modelo de falta declarado explicitamente, nunca existe algo como tolerância genérica a qualquer tipo de problema imaginável. Os modelos habituais organizam-se por ordem crescente de severidade. No modelo de paragem, também chamado crash, o componente simplesmente deixa de responder e nunca mais volta a agir, o caso mais simples de todos. No modelo de omissão, o componente ocasionalmente não responde, mas continua a funcionar corretamente no resto do tempo. No modelo temporal, o componente responde, mas por vezes fora do intervalo de tempo esperado, o que pode ser tão problemático como não responder de todo. E no modelo arbitrário, também chamado bizantino, o componente pode exibir absolutamente qualquer comportamento, incluindo dar respostas incorretas e inconsistentes entre si a diferentes observadores. Quanto mais severo for o modelo de falta que se quer tolerar, mais dispendioso, em recursos e em complexidade, se torna o mecanismo necessário para o tolerar de facto.

Quatro formas de lidar com faltas

- **Prevenção de faltas** – impedir a sua introdução (processos de desenho, revisão, normas).
- **Tolerância a faltas** – assegurar serviço correto na presença de faltas (redundância, detecção e recuperação).
- **Remoção de faltas** – reduzir número e severidade (verificação, teste, diagnóstico).
- **Previsão de faltas** – estimar incidência e consequências (modelos, análise estatística).
- Uma arquitetura séria emprega os quatro. Concentrar o esforço apenas na tolerância é um erro comum e dispendioso.

Existem quatro estratégias distintas, e complementares entre si, para lidar com faltas num sistema. A prevenção de faltas tenta impedir a sua introdução logo à partida, através de processos de desenho cuidadoso, revisão de código e cumprimento de normas de engenharia. A tolerância a faltas assegura serviço correto mesmo na presença de faltas já existentes, tipicamente através de redundância, detecção ativa e recuperação automática. A remoção de faltas tenta reduzir o número e a severidade das faltas já introduzidas, através de verificação formal, testes sistemáticos e diagnóstico cuidadoso. E a previsão de faltas tenta estimar antecipadamente a sua incidência esperada e as suas consequências prováveis, através de modelos matemáticos e análise estatística de dados históricos. Uma arquitetura de infraestrutura séria emprega as quatro estratégias em conjunto. Concentrar todo o esforço apenas na tolerância a faltas, ignorando as outras três, é um erro comum na indústria, e costuma sair caro a prazo.

Quando a redundância não chega

- A tolerância a faltas assenta em redundância – de componentes, de informação ou de tempo.
- A sua eficácia depende de uma hipótese frequentemente implícita: a **independência das faltas**.
- Réplicas que partilham alimentação, rede, versão de software ou defeito de desenho falham em conjunto. A redundância aparente não se traduz em tolerância real.
- **Falta de modo comum**: aquela que afeta simultaneamente todas as réplicas. É o limite fundamental de qualquer esquema de redundância.

A tolerância a faltas assenta, no fundo, em redundância, seja ela redundância de componentes físicos, redundância de informação armazenada, ou redundância de tempo disponível para repetir uma operação que falhou. A eficácia real desta redundância depende, contudo, de uma hipótese que é frequentemente implícita e raramente questionada: a independência das faltas entre as várias réplicas. Réplicas que partilham a mesma fonte de alimentação elétrica, a mesma ligação de rede, a mesma versão de software, ou o mesmo defeito de desenho original, tendem a falhar todas ao mesmo tempo, em conjunto. Nesses casos, a redundância que parecia existir no papel não se traduz em tolerância a falhas real na prática. A este fenómeno chama-se falta de modo comum, aquela que afeta simultaneamente todas as réplicas de uma vez, e representa o limite fundamental e incontornável de qualquer esquema de redundância, por mais réplicas que se acrescentem.

SLI, SLO e SLA

- **SLI** (*Service Level Indicator*) – uma grandeza efetivamente medida, com definição operacional precisa.
- **SLO** (*Service Level Objective*) – um valor de referência para esse indicador, assumido internamente.
- **SLA** (*Service Level Agreement*) – um compromisso contratual, com consequências definidas em caso de incumprimento.
- A ordem importa: sem indicador não há objetivo verificável, e sem objetivo o acordo é inexecutável.
- Confundir os três é uma das origens mais comuns de conflito entre equipes técnicas e a organização.

Estes três acrônimos formam também eles uma cadeia lógica, semelhante à de falta, erro e falha, mas aplicada ao domínio dos acordos de nível de serviço. Um SLI, *Service Level Indicator*, é uma grandeza que é efetivamente medida no sistema real, com uma definição operacional precisa e sem ambiguidade, por exemplo a percentagem de pedidos que recebem resposta em menos de duzentos milissegundos. Um SLO, *Service Level Objective*, é um valor de referência para esse mesmo indicador, assumido internamente pela equipe como meta a atingir. E um SLA, *Service Level Agreement*, é um compromisso contratual formal, assumido perante um cliente externo, com consequências explícitas definidas em caso de incumprimento. A ordem entre os três importa e não é arbitrária: sem um indicador bem definido não existe um objetivo que se possa verificar de forma objetiva, e sem um objetivo verificável o acordo contratual torna-se, na prática, inexecutável. Confundir estes três conceitos entre si é uma das origens mais comuns de conflito entre equipes técnicas e o resto da organização.

Error budget

- Um SLO inferior a 100% define, por complemento, uma quantidade admissível de indisponibilidade: o **orçamento de erro**.
- Um objetivo de 99,9% num trimestre corresponde a um orçamento de aproximadamente 2,2 horas nesse período.
- A sua função é converter uma discussão qualitativa – «devemos arriscar esta alteração?» – numa decisão com critério explícito.
- Enquanto houver orçamento, a mudança é admissível; esgotado o orçamento, a prioridade desloca-se para a estabilização.
- Torna visível que fiabilidade e velocidade de mudança são grandezas em tensão, e não objetivos independentes.

Um SLO que seja inferior a cem por cento define, por simples complemento aritmético, uma quantidade admissível de indisponibilidade ao longo do período considerado, a que se chama orçamento de erro. Um objetivo de 99,9% ao longo de um trimestre corresponde a um orçamento de aproximadamente duas horas e vinte minutos de indisponibilidade nesse período inteiro. A função prática deste orçamento é transformar uma discussão qualitativa e difícil de resolver, do tipo devemos arriscar esta alteração ou não, numa decisão com um critério explícito e objetivo. Enquanto ainda houver orçamento de erro por gastar, a mudança é considerada admissível. Assim que o orçamento se esgota, a prioridade da equipa desloca-se automaticamente para a estabilização do sistema, e novas mudanças arriscadas ficam suspensas até o orçamento se recompor. Esta ferramenta torna visível algo que, de outra forma, ficaria implícito: fiabilidade e velocidade de mudança são duas grandezas genuinamente em tensão uma com a outra, e não dois objetivos que se possam perseguir de forma independente.

Atributos que competem entre si

- Atributos de qualidade raramente são independentes; melhorar um degrada tipicamente outro.
 - Consistência forte entre réplicas custa latência.
 - Isolamento forte entre consumidores custa eficiência de utilização.
 - Redundância custa recursos e complexidade operacional.
 - Segurança custa, quase sempre, conveniência.
- Não há arquitetura ótima em abstrato – há arquiteturas adequadas a uma ponderação declarada de requisitos.
- **Explicitar a ponderação é parte do trabalho de engenharia**, não um preliminar dele.

Os atributos de qualidade raramente são independentes uns dos outros, e melhorar um deles tipicamente degrada algum outro em troca, quase nunca se ganha tudo ao mesmo tempo sem qualquer custo associado. Consistência forte entre réplicas custa latência adicional, porque é preciso coordenar todas antes de confirmar uma operação. Isolamento forte entre consumidores diferentes custa eficiência na utilização dos recursos disponíveis. Redundância custa mais recursos físicos e mais complexidade operacional do dia a dia. E segurança custa, quase sempre, alguma conveniência para quem usa o sistema. A conclusão que decorre destas trocas constantes é que não existe uma arquitetura ótima em abstrato, existem apenas arquiteturas adequadas a uma ponderação de requisitos que foi declarada explicitamente antes de se desenhar o sistema. Explicitar essa ponderação de requisitos é, ela própria, parte do trabalho de engenharia, não um preliminar dispensável que se possa saltar antes de chegar ao trabalho a sério.

Parte 6

O fio condutor do semestre

A sexta e última parte da aula recua um passo e mostra como todos os blocos do programa desta unidade curricular se encadeiam uns nos outros ao longo do semestre, e que perguntas, mais do que que tecnologias específicas, vão atravessar esse percurso inteiro.

Como os blocos se encadeiam

- **C1** estabelece o substrato: recursos finitos, restrições físicas, modelos de operação.
- **C2** introduz o mecanismo que permite parti-los: virtualização e contentorização.
- **C3** trata da gestão em escala: orquestração, rede programável, automatização.
- **C4** passa dos recursos aos serviços: disponibilidade, escala, desacoplamento.
- **C5** trata do estado, que é o que resiste à abstração: armazenamento.
- **C6** fecha o ciclo: observar e administrar o que foi construído.

O programa da unidade curricular está organizado em seis blocos, identificados de C1 a C6, e cada um parte diretamente daquilo que o anterior deixou por resolver. C1 estabelece o substrato de toda a disciplina: recursos finitos, restrições físicas reais e os modelos de operação que vimos nesta mesma aula. C2 introduz o mecanismo que permite efetivamente partir esses recursos entre consumidores, a virtualização e a containerização. C3 trata da gestão desses recursos partidos já em escala, orquestração de containers, rede programável e automatização de configuração. C4 desloca-se dos recursos individuais para os serviços que correm sobre eles, disponibilidade, escala e desacoplamento entre componentes. C5 trata daquilo que resiste com mais teimosia à abstração de todos os conceitos anteriores, o estado que é preciso armazenar e não perder, através de sistemas de armazenamento. E C6 fecha o ciclo completo, voltando a observar e a administrar tudo aquilo que foi construído ao longo do semestre inteiro.

Perguntas que vão reaparecer

- Que recurso está a ser partilhado, e por que mecanismo?
- Que abstração está a ser oferecida, e em que condições deixa de se sustentar?
- Que modelo de falta é tolerado, e a que custo?
- O estado desejado é declarado ou construído por uma sequência de ações?
- Que indicador permite verificar que o sistema está a fazer o que se pretende?
- Reconhecer estas questões em contextos novos é, mais do que memorizar tecnologias, o objetivo da UC.

Esta última página resume, em forma de perguntas, aquilo que verdadeiramente interessa reter desta primeira aula. Que recurso está a ser partilhado em cada situação concreta, e por que mecanismo é que essa partilha acontece. Que abstração está a ser oferecida a quem usa o sistema, e em que condições precisas é que essa abstração deixa de se sustentar. Que modelo de falta está a ser tolerado, e a que custo real isso é conseguido. O estado desejado do sistema está a ser declarado diretamente, ou está a ser apenas construído por uma sequência de ações que alguém executou. E que indicador concreto permite verificar, de forma objetiva, que o sistema está de facto a fazer aquilo que se pretendia que fizesse. Reconhecer estas mesmas perguntas em contextos completamente novos, à medida que forem aparecendo ao longo do semestre, é, mais do que memorizar o nome de tecnologias específicas, o verdadeiro objetivo desta unidade curricular.